

Confidential

레이다 SW 학습 과제 3

표적 궤적 모의

Target Trajectory Simulation

Janghwan Kim (Harley)

발표 대본

표지 · 약 20초

안녕하십니까. 과제 3, 표적 궤적 모의 발표를 시작하겠습니다.

내용은 세 덩어리입니다. 표적을 어떤 구조체로 정의했는지, 그 표적을 시간에 따라 어떻게 움직였는지, 그리고 그 결과가 맞다는 것을 어떻게 확인했는지입니다.

INDEX

표적 궤적 모의
TargetSim

Janghwan Kim (Harley)

01

표적 구조체 설계

상태 벡터와 좌표계
기동을 $G \cdot \text{축} \cdot \text{구간}$ 으로

5 ~ 9

02

표적 궤적 모의 구현

운동학 모델과 회전행렬
속도 변환과 중점법 적분

10 ~ 14

03

시뮬레이션 결과 및 검증

명세 조건 60 s 결과
세 갈래 검증과 시연

15 ~ 18

2

발표 대본

목차 · 약 35초

1장은 구조체 설계입니다. 표적 하나의 상태를 어떤 변수로 잡을지, 그리고 "기동"이라는 말을 코드가 다룰 수 있는 값 네 개로 어떻게 바꿨는지를 봅니다.

2장이 구현입니다. 운동학 모델을 세우고, 과제에서 추가로 고민해 보라고 한 항목인 속도 변환과, 0.1초를 전진시키는 수치적분을 다룹니다.

3장이 결과와 검증입니다. 그림이 그럴듯하게 나왔다는 것만으로는 맞다고 할 수 없어서, 세 가지 방법으로 따로 확인했습니다.

들어가며

과제가 요구한 것

「과제 3 내용」의 요구사항을 네 덩어리로 묶고, 발표도 그 순서로 갑니다.

1 표적 구조체 정의

표적 최대 10 · 기동 최대 30.
위치는 위도 · 경도 · 고도,
속도는 동체 속도 한 개.
기동은 G · 축 · 시작 · 종료.

01 장

2 표적 상태 갱신

위치와 속도를 ECEF 에서 갱신.
추가 고민 — 동체 속도를 ECEF
Vx, Vy, Vz 로 바꾸는 방법.

02 장

3 시뮬레이션 조건

60 s · 0.1 s 간격.
플랫폼 1 + 대함 1 + 대공 1.
제공받은 좌표변환 코드 사용.

03 장

4 결과 제시

궤적을 LLA 로 출력하고 그림으로.
구조체 · 구현 · 결과를
코드와 함께 설명.

03 장

표적 모의는 실장비 없이 데이터처리 입력을 공급하는 일입니다 — 같은 규격, 반복 가능, 그리고 내가 넣은 값이 곧 참값입니다.
좌표변환과 행렬 연산은 제공받은 코드를 무수정으로 썼고, 새로 작성한 것은 TargetSim.c 하나입니다.

3

발표 대본

도입 · 약 60초

과제 문서의 요구사항을 네 덩어리로 묶었습니다.

첫째가 구조체 정의입니다. 표적은 최대 10개, 표적마다 기동은 최대 30개. 위치는 위도 경도 고도로, 속도는 동체 속도 하나로 받고, 기동에는 G 값과 회전축, 시작과 종료 시각이 들어가야 합니다.

둘째가 상태 갱신인데, ECEF 좌표계에서 하라고 되어 있습니다. 여기에 괄호로 "동체 속도를 ECEF 성분으로 바꾸는 방법을 고민해 보라, 안 해도 된다" 는 항목이 붙어 있습니다. 그런데 이건 안 풀면 구현 자체가 안 되는 부분이라 정면으로 다뤘습니다.

셋째가 시뮬레이션 조건, 넷째가 결과 제시입니다.

아래 띠에 두 가지를 적어 뒀습니다.

하나. 표적 모의가 하는 일은 실장비 없이 데이터처리 입력을 공급하는 것입니다. 실제 해상 시험은 배와 비행기를 띄워야 하고, 같은 상황을 두 번 만들 수 없고, 표적이 정확히 어디 있었는지 알려면 기준 장비가 또 필요합니다. 모의는 제가 넣은 값이 곧 정답이라 그런 문제가 없습니다.

둘. 좌표변환과 행렬 연산은 제공받은 코드를 한 줄도 고치지 않고 그대로 썼습니다. 새로 작성한 것은 TargetSim.c 하나입니다.

들어가며

만든 것 — 계산과 화면을 나눈다

연산 엔진만 따로 빌드해 검증할 수 있게 했습니다. 오늘 숫자는 모두 그렇게 뽑은 것입니다.



4

발표 대본

도입 · 약 50초

만든 물건은 두 계층입니다.

아래 연산 계층이 순수 C 로 된 TargetSimCore 입니다. 이 중에서 CoordinateTransform 과 matrixCalcLib 는 제공받은 코드 원본이고, 제가 새로 쓴 것은 맨 위 TargetSim.c 하나입니다.

위 표현 계층이 MFC 로 만든 화면입니다.

둘 사이에 오가는 것은 오른쪽 구조체 두 개뿐입니다. 설정을 담은 ST_SimConfig 를 넣으면 시각마다의 상태인 ST_SimSample 이 나옵니다. 공개 함수는 여섯 개고, UI 는 Core 안을 전혀 모릅니다.

이렇게 나눈 덕을 실제로 봤습니다. 이 발표 자료의 숫자를 뽑을 때 리눅스에서 gcc 로 Core 만 따로 컴파일해서 돌렸는데, MFC 쪽은 손댈 필요가 없었습니다.

01. 표적 구조체 설계

표적의 상태와 좌표계

표적 하나는 아홉 개 성분으로 정해집니다. 그 값들이 어느 좌표계의 값인지부터 맞춥니다.



5

발표 대본

구조체 · 약 85초

이 장은 두 부분입니다. 위가 좌표계, 아래가 상태 벡터입니다.

왼쪽 위가 ECEF 입니다. 지구 중심이 원점이고 Z 축이 자전축, X 축이 본초자오선 방향입니다. 지구에 박혀 있어 지구와 같이 돕니다. 같은 점을 각도로 적으면 위도 경도 고도, LLA 가 됩니다. 파란 호가 위도, 초록 호가 경도, 빨간 점선이 고도입니다.

가운데가 표적 있는 자리를 확대한 것입니다. 발밑에 접평면을 깔고 북, 동, 아래를 축으로 잡은 것이 NED 이고, 그 위 빨간 축이 동체 좌표계입니다. 기수 방향과 북쪽 사이 각이 Yaw 입니다.

오른쪽이 자세각 셋입니다. Roll 은 날개가 기울고, Pitch 는 기수가 돌리고, Yaw 는 기수가 좌우로 돕니다. 그 아래 식이 동체에서 NED 로 가는 회전행렬인데, 곱하는 순서가 Yaw, Pitch, Roll 로 정해져 있습니다. 순서를 바꾸면 다른 자세가 됩니다.

아래로 내려가겠습니다. 왼쪽이 상태 벡터입니다. ECEF 위치 셋, ECEF 속도 셋, 자세각 셋. 합쳐서 아홉 개입니다.

오른쪽이 명세가 주는 입력이 이 상태로 들어가는 길입니다. 위경고도는 좌표변환 한 번으로 ECEF 위치가 되고, 자세각은 그대로 들어갑니다.

문제는 세 번째 줄입니다. 명세는 속력 하나만 주는데 상태에는 속도 세 성분이 필요합니다. 이 한 줄을 채우는 게 과제의 추가 고민 항목이고, 2장에서 자세히 다루겠습니다.

01. 표적 구조체 설계

기동을 네 개의 값으로

"12 초부터 16 초까지 오른쪽으로 8G" 라는 지시가 구조체 한 줄이 되고, G 하나가 회전 속도를 정합니다.

기동 1건 = 값 네 개

enTurnType	0 없음 / 1 Roll / 2 Yaw / 3 Pitch
gravityValue	크기는 세기, 부호는 회전 방향 [G]
startTime	이 시각부터 (포함) [s]
endTime	이 시각까지 (제외) [s]

$t \in [t_{start}, t_{end})$ 끝과 다음 시작이 같아도 겹침이 아니다
실제로 겹치면 계산 전에 TGT_ERR_MANEUVER_OVERLAP

G 값 하나가 각속도와 선회 반경을 정한다

$$a_c = \frac{V^2}{R} = V\omega = ng$$

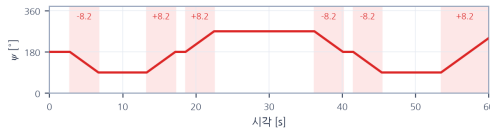
원운동의 구심가속도를 중력가속도의 n 배로 적는다

$$\omega = \frac{ng}{V}, \quad R = \frac{V^2}{ng}$$

$\omega = (gravityValue * G_FORCE) / headingSpeed;$

n [G]	대함 V = 30 m/s	대공 V = 200 m/s
1	18.7 °/s R = 91.8 m	2.8 °/s R = 4079 m
4	74.9 °/s R = 22.9 m	11.2 °/s R = 1020 m
8	149.8 °/s R = 11.5 m	22.5 °/s R = 510 m

기동표 6줄이 실제 응답으로 나타난 모습 (대공 표적)



같은 G 라도 빠른 표적은 덜 꺾인다 — 분모에 속력이 있기 때문이다.

색칠한 구간이 기동표에 적은 (start, end) 이고, 그 안에서만 기울기가 생긴다. 기울기가 곧 ω 다.

기동 지시는 "위도 32.1 에서 좌 90도" 처럼 좌표로 온다. 구조체에는 시각 칸만 있어 Core 를 돌려 그 좌표에 맞는 시각을 찾아 start / end 로 옮겼다.

발표 대본

구조체 · 약 85초

"12초부터 16초까지 오른쪽으로 8G 로 꺾어라" 라는 말을 구조체 한 줄로 바뀌어야 합니다.

왼쪽 위가 그 네 개입니다. 회전축, G 값, 시작 시각, 종료 시각. 축은 명세가 정한 그대로 0이 없음, 1이 Roll, 2가 Yaw, 3이 Pitch 입니다. G 값은 크기가 세기, 부호가 방향입니다.

그 아래 구간 규칙을 하나 정했습니다. 시작은 포함하고 끝은 포함하지 않습니다. 그래야 한 기동이 16초에 끝나고 다음이 16초에 시작해도 겹침으로 잡히지 않습니다. 실제로 겹치는 경우에는 계산을 시작하기 전에 오류 코드로 돌려보냅니다.

오른쪽이 왜 각속도가 아니라 G 로 받는지입니다.

G 가 실제 한계값이기 때문입니다. 전투기는 9G 근처가 한계이고 조종사도 그 근처에서 의식을 잃습니다. 시나리오를 만드는 사람은 "이 표적이 낼 수 있는 G 가 얼마인가" 로 생각하지 "초당 몇 도" 로 생각하지 않습니다.

식은 두 줄입니다. 원운동의 구심가속도는 속력 곱하기 각속도인데, 그게 중력가속도의 n 배라고 하면 각속도는 n 곱하기 9.80665 나누기 V, 선회 반경은 V 제곱 나누기 n 곱하기 9.80665 가 나옵니다. G 값 하나가 회전 속도와 반경을 동시에 정합니다.

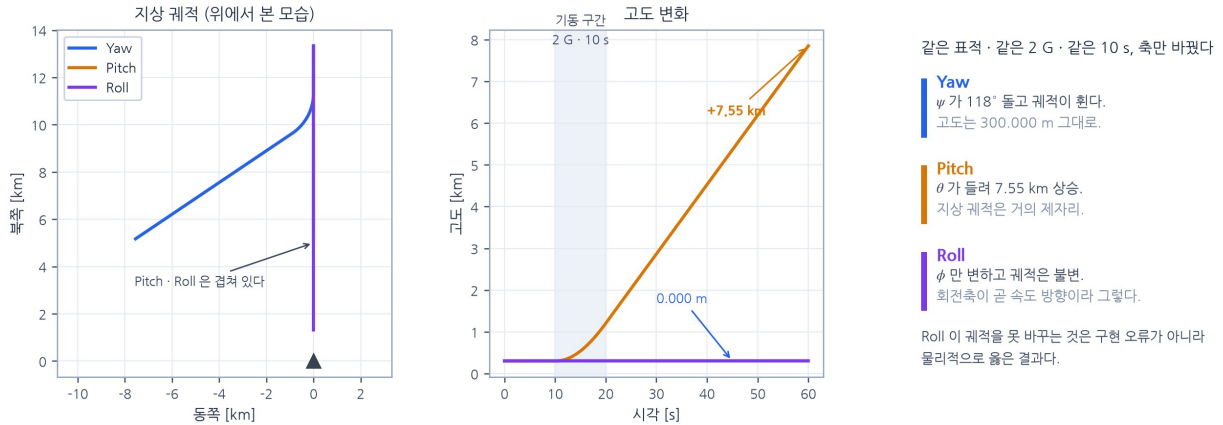
그 아래 표에서 읽으실 것은, 같은 G 라도 빠른 표적이 훨씬 덜 꺾인다는 점입니다. 분모에 속력이 있으니까요. 초속 30미터 대함은 1G 로도 초당 18.7도를 도는데, 초속 200미터 대공은 2.8도밖에 못 둡니다.

아래 왼쪽 그래프가 기동표가 실제 결과로 나타난 모습입니다. 분홍색 구간이 기동표에 적은 구간인데, 거기서만 선이 기울고 나머지는 평평합니다. 기울어진 정도가 방금 본 각속도입니다.

01. 표적 구조체 설계

회전축 세 가지의 응답 — 실측

같은 표적에 같은 2 G 를 10 초씩, 축만 바꿔 걸어 본 결과입니다.



7

발표 대본

구조체 · 약 60초

축이 셋이라고 말로만 하면 감이 안 오니까 실제로 돌려 봤습니다.

같은 대공 표적, 같은 2G, 같은 10초입니다. 축만 바꿨습니다.

왼쪽이 위에서 본 지상 궤적입니다. 파란색 Yaw 는 기수가 옆으로 돌아 궤적이 크게 휘었습니다. 118도 돌았습니다.

가운데가 고도입니다. 주황색 Pitch 가 10초 동안 7.55킬로미터를 올라갔습니다. 대신 위에서 보면 거의 제자리입니다. 파란 Yaw 는 고도가 300.000미터 그대로입니다.

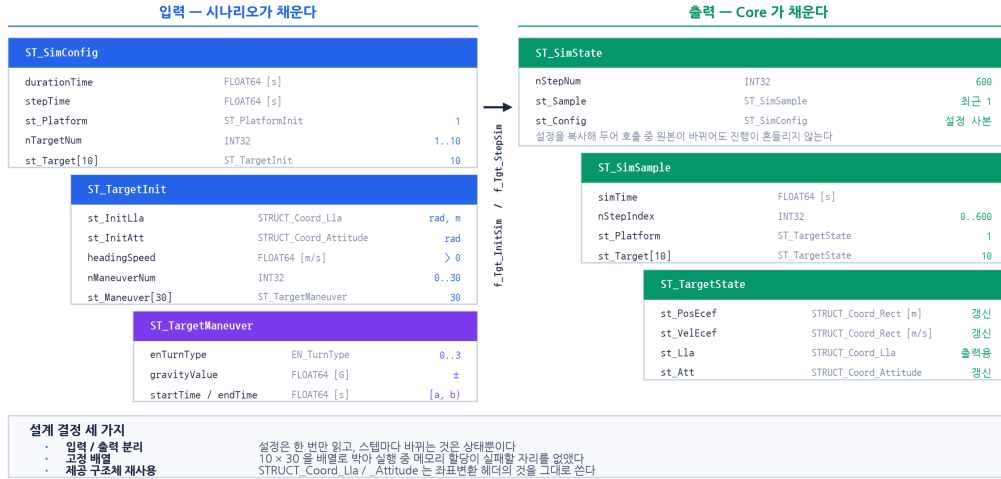
보라색 Roll 이 재미있습니다. 궤적도 고도도 전혀 안 변했습니다. 처음에는 구현이 틀린 줄 알고 한참 봤습니다.

그런데 물리적으로 맞는 결과였습니다. Roll 축이 곧 속도 방향 그 자체이기 때문입니다. 자기가 가는 방향을 축으로 삼아 몸통만 돌리는 거라, 방향이 바뀔 수가 없습니다.

실제 비행기도 Roll 만으로는 방향을 못 바꿉니다. Roll 로 기울인 다음 Pitch 를 당겨야 선회가 됩니다. 저희 모델은 세 축을 독립적으로 적분하니까 그 결합이 없습니다. 이건 모델의 한계이기도 해서 마지막에 다시 언급하겠습니다.

01. 표적 구조체 설계

구조체 관계와 설계 결정



8

발표 대본

구조체 · 약 55초

지금까지 이야기한 것을 구조체로 그리면 이렇게 됩니다.

왼쪽이 입력입니다. 맨 위 ST_SimConfig 가 시뮬레이션 시간과 갱신 간격, 표적 수를 들고, 그 안에 플랫폼 하나와 표적 배열이 들어갑니다. 표적마다 기동 배열이 또 들어갑니다. 10개와 30개는 배열 크기로 박아 뒀습니다.

오른쪽이 출력입니다. ST_SimState 가 진행 상태를 들고, 그 안의 ST_SimSample 이 "이 시각 전원의 상태" 한 장입니다. 그 안에 객체마다 ST_TargetState 가 있고, 거기에 ECEF 위치, ECEF 속도, LLA, 자세각이 들어갑니다.

각 필드 옆에 타입과 단위, 개수를 같이 적어 뒀습니다. 단위를 문서가 아니라 구조체 옆에 적어 두는 게 나중에 헛갈리지 않는 방법이라고 봤습니다.

아래 세 가지가 설계 결정입니다. 입력과 출력을 완전히 갈랐고, 동적 할당 없이 고정 배열을 썼고, 위치와 자세 구조체는 제공받은 좌표변환 헤더의 것을 그대로 썼습니다. 새로 만들면 변환할 때마다 복사해 넣어야 하니까요.

01. 표적 구조체 설계

코드로 보는 구조체

입력 — TargetSim.h

```
// 기동 1건 . gravityValue 의 부호가 회전 방향이다 .
typedef struct
{
    EN_TurnType    enTurnType;    // 0~3
    FLOAT64        gravityValue;  // [G]
    FLOAT64        startTime;     // [s]
    FLOAT64        endTime;       // [s]
} ST_TargetManeuver;

// 위치는 위 / 경 / 고도 , 속도는 동체 x 축 방향 크기 .
typedef struct
{
    STRUCT_Coord_Lla    st_InitLla;
    STRUCT_Coord_Attitude st_InitAtt;
    FLOAT64             headingSpeed; // V_heading
    INT32               nManeuverNum;
    ST_TargetManeuver    st_Maneuver[TGT_MAX_MANEUVER_NUM];
} ST_TargetInit;
```

출력 — TargetSim.h

```
// 위치 · 속도는 ECEF 에서 갱신하고 , LLA 는 출력과
// 다음 스텝의 NED 기준으로 함께 보관한다 .
typedef struct
{
    FLOAT64        simTime;    // [s]
    STRUCT_Coord_Rect st_PosEcef; // [m]
    STRUCT_Coord_Rect st_VelEcef; // [m/s]
    STRUCT_Coord_Lla st_Lla;
    STRUCT_Coord_Attitude st_Att;
} ST_TargetState;

// 한 시각의 플랫폼 · 표적 상태 .
typedef struct
{
    FLOAT64        simTime;
    INT32          nStepIndex;
    ST_TargetState st_Platform;
    ST_TargetState st_Target[TGT_MAX_TARGET_NUM];
} ST_SimSample;
```

구간은 [시작, 종료)

끝과 다음 시작이 같아도 겹침이 아니다.
실제로 겹치면 계산 전에 거부한다.

ECEF 와 LLA 를 함께

갱신은 ECEF, 출력은 LLA, 다음 스텝의
NED 기준도 LLA — 한 번만 변환해 둔다.

플랫폼도 같은 구조체

기동이 없는 표적일 뿐이라 각속도만 0 으로
넣어 같은 경로로 전진시킨다.

9

발표 대본

코드 · 약 60초

실제 헤더입니다. 왼쪽이 입력, 오른쪽이 출력입니다.

왼쪽 위가 기동 한 건인데, 앞에서 본 네 개가 그대로 들어가 있습니다. 그 아래가 표적 초기 설정입니다. 위치가 STRUCT_Coord_Lla, 자세가 STRUCT_Coord_Attitude 인데 제공받은 구조체입니다. 단위가 라디안인 것도 그 헤더를 따랐습니다. 파란 줄 headingSpeed 가 속력 하나입니다.

오른쪽 위가 객체 하나의 상태입니다. 눈여겨보실 것은 파란 두 줄, ECEF 위치와 속도를 들고 있으면서 그 아래 LLA 도 같이 들고 있다는 점입니다.

값이 중복되는 것처럼 보이는데 이유가 셋입니다. 갱신은 ECEF 에서 해야 지구 곡률 향이 사라지고, 출력과 화면은 LLA 라야 읽히고, 다음 스텝의 속도를 만들 때도 그 지점의 위경도가 필요합니다. NED 축을 세우려면 내가 지구 어디에 있는지 알아야 하니까요. 세 군데에서 쓰는데 스텝마다 한 번만 변환해 두면 되니까 들고 있는 쪽이 이득입니다.

아래 카드 셋이 요약입니다. 마지막 카드만 짚으면, 플랫폼도 같은 구조체를 씁니다. 기동이 없는 표적일 뿐이라 각속도만 0 으로 넣고 똑같은 함수를 통과시킵니다. 코드가 두 벌이 안 됩니다.

02. 표적 궤적 모의 구현

운동학 모델과 ECEF 적분

구현을 시작하기 전에 방정식부터 세우고, 어느 좌표계에서 적분할지를 정합니다.

표적 운동을 지배하는 세 줄

$$\dot{\mathbf{p}}^e = \mathbf{v}^e$$

위치의 시간 변화율이 곧 속도

$$\mathbf{v}^e = \mathbf{C}_n^e(q, \lambda) \mathbf{C}_b^n(\phi, \theta, \psi) [V \ 0 \ 0]^T$$

속도는 기수 방향 크기 V 를 두 번 돌려 만든다

$$\dot{\Theta} = \omega(t), \quad \omega_i = n_i g/V$$

자세각은 기동이 걸린 구간에서만 돈다

이 모델이 전제하는 것

운동학 모델 힘이 아니라 속도 · 각속도를 직접 준다.
중력 · 추력 · 항력을 풀지 않아도 표적에는 맞지 않는다.

속력 일정 회전만 하므로 $|v|$ 가 변하지 않는다.
가속이 필요하면 V 를 시간 함수로 바꿔야 한다.

오일러각 적분 자세각을 직접 적분한다.
 $\theta \rightarrow \pm 90^\circ$ 에서 짐벌락 — 사원수로 바꾸면 풀린다.

왜 ECEF 에서 적분하는가

LLA 에서 직접
 $\dot{\phi} = \frac{v_N}{M(q) + h}$
분모가 위도마다 달라
곧게 날려도 고도가 돈다

ECEF 에서
 $\mathbf{p}_{k+1}^e = \mathbf{p}_k^e + \mathbf{v}^e \Delta t$
미터 단위 직교좌표라
그냥 더하면 된다

본 과제 조건에서 실제로 얼마인가

$$d = V \Delta t$$

$$20 \text{ m}$$

$$d^2/2R$$

$$31.4 \mu\text{m}/\text{스텝}$$

$$600 \text{ 스텝}/\text{누적}$$

$$18.8 \text{ mm}$$

$$\text{중점법 보정 후}$$

$$\text{고도 불변}$$

값은 작지만, 이런 항이 남으면 간격을 줄여도 오차가 선형으로만 줄어들어 2차 수렴이 깨진다.

10

발표 대본

구현 · 약 80초

구현을 시작하기 전에 방정식부터 세웠습니다. 위에 세 줄입니다.

첫 줄. 위치의 시간 변화율이 곧 속도다. 정의입니다.

둘째 줄. 속도는 기수 방향 크기 V 를 두 번 돌려 만든다. 회전행렬 두 개가 곱해져 있는데, 오른쪽이 동체에서 NED 로, 왼쪽이 NED 에서 ECEF 로 가는 행렬입니다. 이 줄이 다음 장의 주제입니다.

셋째 줄. 자세각은 기동표가 지시하는 각속도로 돈다. 그 각속도가 앞에서 본 n 곱하기 g 나누기 V 이고, 기동이 걸린 구간에서만 0 이 아닙니다.

아래 왼쪽이 이 모델이 전제하는 것입니다. 이걸 명시적으로 적어 두는 게 중요하다고 봤습니다. 힘이 아니라 속도를 직접 주는 운동학 모델이고, 회전만 하니까 속력이 일정하고, 오일러각을 직접 적분하니까 Pitch 90도 근처에서 짐벌락이 생깁니다.

아래 오른쪽이 왜 ECEF 에서 적분하느냐입니다.

LLA 에서 바로 적분하면 위도 변화율의 분모에 곡률반경이 들어가는데, 그게 위도마다 다릅니다. 매 스텝 다시 구해야 하는 건 둘째 치고, 진짜 문제는 곧게 날려도 고도가 조금씩 올라간다는 겁니다. 지면이 곡면인데 표적은 직선으로 나아가려고 하니까요.

ECEF 는 미터 단위 직교좌표라 그냥 더하면 됩니다. 지구가 둥근 것은 좌표계가 이미 품고 있습니다.

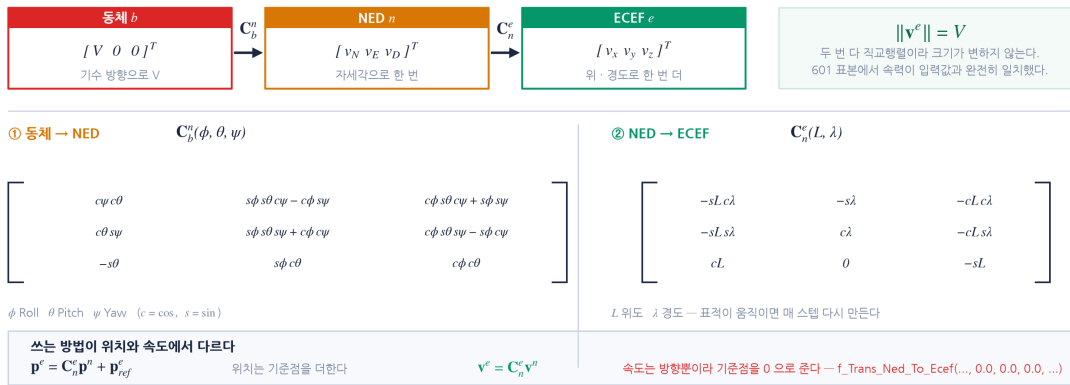
아래 숫자를 보시면, 대공 표적은 0.1초에 20미터를 가고 스텝당 고도 편차가 31마이크로미터, 600스텝이면 18.8 밀리미터입니다. 값 자체는 작습니다. 그런데 중요한 건 크기가 아니라 성질입니다. 이런 항이 남아 있으면 갱신 간격을 줄여도 오차가 선형으로만 줄어들어 2차 수렴이 깨집니다.

02. 표적 궤적 모의 구현

동체 속도를 ECEF 속도로 — 과제의 추가 고민

* 동체 속도 값 ($V_{heading}$) 을 ECEF 좌표계 V_x, V_y, V_z 성분으로 변환하는 방법 * 에 대한 답입니다.

동체 속도를 ECEF 속도로 — 회전 두 번



11

발표 대본

구현 · 약 80초

과제에서 추가로 고민해 보라고 한 항목입니다. 결론부터 말씀드리면 회전 두 번입니다.

위 왼쪽이 출발점입니다. 동체 좌표에서 기수 방향이 x 축이니까 속도는 그냥 $V, 0, 0$ 입니다. 옆도 0, 아래도 0. 이게 제일 쉬운 형태이고, 속도를 크기 하나로 받은 덕입니다.

첫 번째 회전이 동체에서 NED 입니다. 자세각으로 돌리면 북쪽, 동쪽, 아래 성분이 나옵니다. Yaw 가 180도면 북쪽 성분이 마이너스 V 가 나오겠죠. 남쪽으로 간다는 뜻입니다.

두 번째 회전이 NED 에서 ECEF 입니다. 여기에 그 지점의 위도와 경도가 들어갑니다. 같은 "북쪽" 이라도 제주 앞 바다에서의 북쪽과 뉴욕에서의 북쪽은 ECEF 에서 전혀 다른 방향이니까요.

오른쪽 초록 상자가 이 방식의 이점입니다. 두 번 다 회전행렬이고, 회전행렬은 직교행렬이라 벡터의 크기를 바꾸지 않습니다. 그래서 결과 속도의 크기는 처음 넣은 V 와 정확히 같습니다. 증명된 성질이지 시험으로 알아내는 게 아닙니다. 다만 구현이 그 성질을 지키는지는 확인해야 해서, 뒤에서 601개 표본의 속력을 켜 결과를 보여 드리겠습니다.

아래가 두 행렬을 펼친 것입니다. 외우실 필요는 없고 구조만 보시면 됩니다. 왼쪽은 자세각 세 개의 사인 코사인이 섞여 있고, 오른쪽은 위도와 경도만 들어갑니다. 표적이 움직이면 위경도가 바뀌니까 오른쪽 행렬은 매 스텝 다시 만들어야 합니다.

맨 아래가 제가 한 번 틀렸던 자리입니다. 위치를 옮길 때는 회전한 다음 기준점을 더합니다. 그런데 속도는 방향뿐이라 기준점을 더하면 안 됩니다. 제공받은 함수는 위치용이라 평행이동이 들어 있어서, 기준점 자리에 0 을 세 개 넣어 회전만 남겼습니다.

02. 표적 궤적 모의 구현

코드 ① — 동체 속도를 ECEF 속도로

TargetSimCore/src/TargetSim.c

```
// 동체 속도 (V, 0, 0) 을 자세로 한 번, 위경도로 한 번 돌려 ECEF 속도로 만든다.
// 회전만 거치므로 속력은 그대로 보존된다.
static STRUCT_Coord_Rect f_Tgt_GetVelEcef(const STRUCT_Coord_Attitude *st_Att,
const STRUCT_Coord_Lla *st_Lla, FLOAT64 headingSpeed)
{
    STRUCT_Coord_Rect st_VelNed;

    // 참고 소스의 f_Trans_Body_To_Ned 는 인자를 Roll, Yaw, Pitch 순서로 받는다.
    st_VelNed = f_Trans_Body_To_Ned(headingSpeed, 0.0, 0.0,
    st_Att->Roll, st_Att->Yaw, st_Att->Pitch);

    // f_Trans_Ned_To_Ecef 는 기준점 이동까지 더한다. 방향만 옮길 때는 기준점을 원점으로 준다.
    return f_Trans_Ned_To_Ecef(st_VelNed.x, st_VelNed.y, st_VelNed.z,
    0.0, 0.0, 0.0, st_Lla->Lat, st_Lla->Lon);
}
```

기준점을 0 으로 주는 이유

f_Trans_Ned_To_Ecef 는 회전에 더해
기준점만큼 평행이동까지 한다. 속도는
"위치"가 아니라 "방향"이라 빼야 한다.

인자 순서를 주석에 남긴 이유

제공받은 함수가 Roll, Yaw, Pitch 순서로
받는다. 자세각 구조체 필드 순서와 달라
한 번 틀렸던 자리다.

속력 보존은 성질, 시험은 확인

회전행렬은 직교행렬이라 크기를 바꾸지
않는다. 601 표본에서 속력이 소수점
12 자리까지 입력값과 일치했다.

12

발표 대본

코드 · 약 55초

방금 설명한 게 코드로는 실행문 두 줄입니다.

첫 줄이 동체에서 NED 입니다. headingSpeed 를 첫 인자로 주고 나머지 둘을 0 으로 줍니다. 이게 "기수 방향으로 V" 라는 뜻입니다.

둘째 줄이 NED 에서 ECEF 인데, 기준점 자리에 0 이 세 개 들어가 있는 게 보이실 겁니다. 방금 말씀드린 그 부분입니다.

아래 카드 셋이 이 함수에서 신경 쓴 것입니다.

왼쪽이 기준점 이야기입니다.

가운데가 인자 순서입니다. 제공받은 함수가 Roll, Yaw, Pitch 순서로 받는데 구조체 필드는 Roll, Pitch, Yaw 순서입니다. 컴파일도 되고 실행도 되는데 결과만 틀리는 종류의 실수라, 주석을 남겨 뒀습니다.

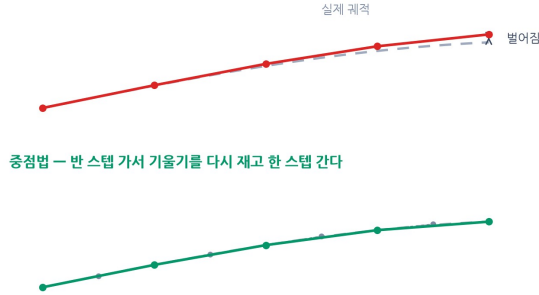
오른쪽이 속력 보존 확인입니다. 601개 표본에서 속력이 소수점 열두 자리까지 입력값과 같았습니다.

02. 표적 궤적 모의 구현

0.1 초를 어떻게 전진시킬 것인가

어느 지점의 기울기를 쓰느냐가 오차 차수를 가릅니다.

오일러법 — 출발할 때의 기울기로 한 스텝 간다



굽은 길을 직선 토막으로 잇는 것은 같다. 다른 것은 어느 지점의 기울기를 쓰느냐다 — 중점의 기울기가 구간 평균에 훨씬 가깝다.

두 방법의 식과 오차 차수

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta t f(t_k, \mathbf{x}_k)$$

국소 $O(\Delta t^2)$ · 전역 $O(\Delta t)$ — 1차

$$\mathbf{k}_I = f(t_k, \mathbf{x}_k)$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta t f\left(t_k + \frac{\Delta t}{2}, \mathbf{x}_k + \frac{\Delta t}{2} \mathbf{k}_I\right)$$

국소 $O(\Delta t^3)$ · 전역 $O(\Delta t^2)$ — 2차

이 과제에서의 반 스텝

자세각	각속도가 상수라 반 스텝 값을 바로 더한다
위치	스텝 시작 속도로 반 스텝 예측한다
속도	그 자리 · 그 자세에서 다시 만든다

계산량은 좌표변환 2회 → 4회로 늘지만, 0.1 s 에서 오차가 1.36 m → 1.36 cm 로 100 배 줄었다.

13

발표 대본

구현 · 약 60초

위치와 속도를 구했으면 이제 0.1초를 전진시켜야 합니다.

왼쪽 위가 제일 단순한 방법입니다. 지금 속도로 0.1초 곱해서 더한다. 오일러법입니다.

직진이면 문제가 없습니다. 그런데 선회 중이면 0.1초 동안에도 기수가 계속 도는데 출발할 때의 방향으로 쪽 밀어버리니까, 굽은 길을 직선 토막으로 근사하게 됩니다. 늘 바깥쪽으로 밀립니다. 오른쪽 끝에 "떨어짐" 이라고 표시한 부분입니다.

왼쪽 아래가 중점법입니다. 일단 반 걸음만 가봅니다. 회색 점선이 그겁니다. 거기서 기울기를 다시 재고, 그 방향으로 한 걸음을 갑니다. 반 걸음 간 자리의 기울기가 구간 평균에 훨씬 가깝습니다.

오른쪽이 식과 오차 차수입니다. 오일러법은 전역 오차가 델타티에 비례하는 1차, 중점법은 델타티 제곱에 비례하는 2차입니다. 2차라는 말은 간격을 절반으로 줄이면 오차가 4분의 1이 된다는 뜻입니다. 뒤에서 숫자로 확인하겠습니다.

계산량은 좌표변환 두 번에서 네 번으로 늘어납니다. 대신 0.1초에서 오차가 1.36미터에서 1.36센티미터로 100배 줄었습니다. 치를 만한 값이라고 봤습니다.

02. 표적 궤적 모의 구현

코드 ② — 한 스텝 전진 (중점법)

TargetSimCore/src/TargetSim.c f_Tgt_Propagate (발해)

```

halfStep = 0.5 * stepTime;

// ① 반 스텝 뒤의 자세 — 각속도가 구간 내 상수라 바로 더한다
st_AttMid.Roll = st_State->st_Att.Roll + (st_Rate->Roll * halfStep);
st_AttMid.Pitch = st_State->st_Att.Pitch + (st_Rate->Pitch * halfStep);
st_AttMid.Yaw = st_State->st_Att.Yaw + (st_Rate->Yaw * halfStep);

// ② 반 스텝 뒤의 위치 — 스텝 시작 속도로 예측
st_VelStart = f_Tgt_GetVelEcef(&st_State->st_Att, &st_State->st_Lla, headingSpeed);
st_PosMid.x = st_State->st_PosEcef.x + (st_VelStart.x * halfStep);
...

// ③ 그 자리 · 그 자세에서 구간 속도가 이 스텝의 대표 속도
st_LlaMid = f_Trans_Ecef_To_Lla(st_PosMid.x, st_PosMid.y, st_PosMid.z);
st_VelMid = f_Tgt_GetVelEcef(&st_AttMid, &st_LlaMid, headingSpeed);

// ④ 그 속도로 한 스텝을 옮긴다
st_Next.st_PosEcef.x = st_State->st_PosEcef.x + (st_VelMid.x * stepTime);
...

```

② 를 빼먹었을 때 생긴 일

자세만 반 스텝 돌리고 위치를 그대로 두면, 표적은 앞으로 나아갔는데 "북쪽"의 방향은 뒤에 남는다. 그 차이가 매 스텝 $d/2R$ 만큼 고도로 쌓인다.

계산이 깨져도 상태는 지킨다

사본에서 전부 계산하고 유효성을 확인한 뒤 모두 성공했을 때만 한 번에 반영한다.

알고도 남겨 둔 한계 — 짐벌락

Pitch 가 $\pm 90^\circ$ 근처면 Yaw 축이 속도 방향과 겹쳐 Yaw 기동이 궤적을 바꾸지 못한다.

14

발표 대본

코드 · 약 70초

중점법이 코드로는 이렇게 생겼습니다. 네 덩어리입니다.

- ①번. 자세각을 반 스텝 돌립니다. 각속도가 구간 안에서 상수라 그냥 더하면 됩니다.
- ②번. 반 스텝 뒤의 위치를 구합니다. 스텝 시작 속도로 예측합니다.
- ③번. 그 자리, 그 자세에서 속도를 다시 구합니다. 이게 이 스텝의 대표 속도입니다.
- ④번. 그 속도로 한 스텝을 옮깁니다.

오른쪽 첫 카드가 ②번 이야기인데, 여기가 제가 처음에 빼먹었던 부분입니다.

원래는 자세만 반 스텝 돌리고 위치는 그대로 두었습니다. 겉보기에는 잘 돌았는데, 수평으로 날린 표적의 고도가 스텝마다 조금씩 올라갔습니다. 스텝 시작 지점의 위경도로 NED 축을 세우면, 표적은 이미 앞으로 나아갔는데 "북쪽"의 방향은 뒤에 남아 있는 셈이 됩니다. 앞 장에서 본 d 제곱 나누기 $2R$ 이 바로 이 항입니다. 위치까지 반 스텝 옮기고 나서 사라졌습니다.

가운데 카드가 방어적 구현입니다. 제공받은 변환 함수들은 실패를 알려주지 않고 NaN 이나 무한대를 그냥 리턴합니다. 그래서 사본에서 전부 계산하고, 결과가 유한한지 확인하고, 다 통과했을 때만 한 번에 반영합니다. 그러면 "표적 1은 새 시각인데 표적 2는 옛 시각" 같은 상태가 아예 안 생깁니다.

아래 카드가 알고도 남겨 둔 한계, 짐벌락입니다. 사원수를 쓰면 풀리는데 과제 범위를 넘는다고 보고 주석으로 명시했습니다.

03. 시뮬레이션 결과 및 검증

시뮬레이션 조건

「과제 3 내용」 2항의 값을 그대로 넣었습니다.

	위도 [°]	경도 [°]	고도 [m]	속력 [m/s]	Roll [°]	Pitch [°]	Yaw [°]
플랫폼	32.0	126.0	0	0	0	0	0
대함 표적	32.125	126.03	0	30	0	0	270 (서)
대공 표적	32.12	126.0	300	200	0	0	180 (남)

시간 60s · 간격 0.1s

스텝 600 회, 표본 601 개.
시각은 누적이 아니라 스텝 번호 × 간격으로
매겨 반올림 오차가 쌓이지 않게 했다.

검증하기 좋은 조건이다

기동이 없으므로 답을 미리 알 수 있다.
대함은 정서진 → 위도 불변,
대공은 정남진 → 경도 불변, 둘 다 고도 불변.

플랫폼 속력이 0 인 점

각속도 계산에 속력이 분모로 들어가지만,
그 계산은 기동 조회 안에만 있다.
플랫폼은 기동이 없어 그 자리를 지나가지 않는다.

15

발표 대본

결과 · 약 50초

과제 문서 2항의 값을 그대로 넣었습니다.

플랫폼은 위도 32도, 경도 126도, 고도 0, 속력 0. 움직이지 않습니다.

대함 표적은 위도 32.125, 경도 126.03, 고도 0, 초속 30미터, Yaw 270도. 서쪽입니다.

대공 표적은 위도 32.12, 경도 126.0, 고도 300미터, 초속 200미터, Yaw 180도. 남쪽입니다.

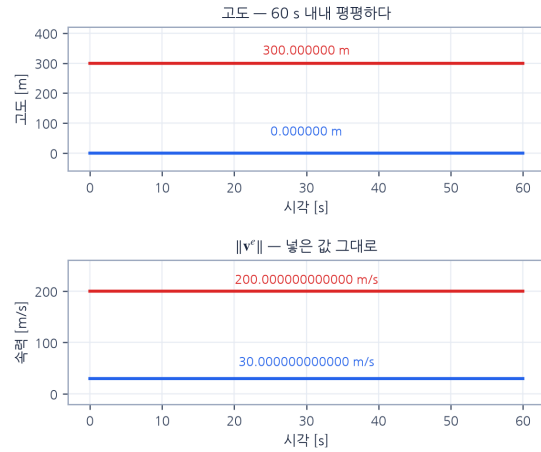
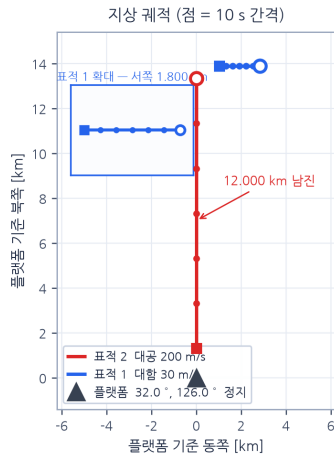
60초에 0.1초 간격이니까 스텝이 600번, 표본이 601개입니다. 시각은 누적해서 더하지 않고 스텝 번호에 간격을 곱해서 매겼습니다. 0.1을 600번 더하면 반올림 오차가 쌓이는데, 곱셈은 한 번이라 안 쌓입니다.

가운데 카드가 중요합니다. 이 조건은 검증하기 아주 좋습니다. 기동이 없으니 답을 미리 알 수 있습니다. 대함은 정서쪽으로만 가니 위도가 변하면 안 되고, 대공은 정남쪽으로만 가니 경도가 변하면 안 되고, 둘 다 수평이니 고도가 변하면 안 됩니다.

03. 시뮬레이션 결과 및 검증

결과 ① — 60 초 궤적

플랫폼을 원점으로 잡은 동-북 평면입니다. 점 하나가 10 초입니다.



16

발표 대본

결과 · 약 55초

결과입니다. 플랫폼을 원점으로 잡은 동-북 평면이고, 점 하나가 10초입니다.

빨간색 대공 표적이 정남쪽으로 12킬로미터를 내려갔습니다. 초속 200미터 곱하기 60초니까 맞습니다.

파란색 대함 표적은 너무 작아서 왼쪽 위에 확대해서 넣었습니다. 서쪽으로 1.8킬로미터입니다.

둘 다 완벽한 직선입니다. 기동을 안 걸었으니 그래야 합니다.

오른쪽 위가 고도입니다. 60초 내내 평평합니다. 대공은 300.000000미터, 대함은 0.000000미터입니다.

오른쪽 아래가 ECEF 속도의 크기입니다. 소수점 열두 자리까지 넣은 값 그대로입니다. 앞에서 회전형렬이 크기를 안 바꾼다고 말씀드린 그 성질이 그대로 나온 겁니다.

여기까지가 눈으로 보는 확인이고, 다음 장이 숫자로 하는 확인입니다.

03. 시뮬레이션 결과 및 검증

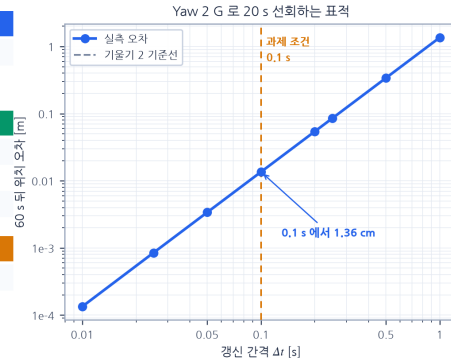
결과 ② — 세 가지 방법으로 검증

각각이 다른 종류의 실수를 잡습니다 — 스케일, 방향, 그리고 적분 방법.

세 가지 방법으로 검증했다

① 해석해와 대조		
스케일이 틀린 경우를 잡는다		
대함 이동거리	$30 \times 60 = 1800 \text{ m}$	$+1.3\text{e-}08 \text{ m}$
대공 이동거리	$200 \times 60 = 12000 \text{ m}$	$+1.7\text{e-}07 \text{ m}$
② 불변량 보존		
방향이 틀린 경우를 잡는다		
고도 (수평 비행)	300 m 유지	$30.8 \mu\text{m}$
속력 (회전만)	200 m/s 유지	0 (601 표본 일치)
위도 · 경도 (적진)	정서진 · 정남진	$0.1 \mu\text{m} / 0$
③ 수렴 차수		
적분 방법이 틀린 경우를 잡는다		
Δt 를 1/2 로	오차 1/4 (2차)	기울기 2.002

이동거리는 601개 표본 사이 거리를 전부 더해서 얻은 값이다 — 직선거리가 아니라 실제로 지나간 길이. 12 km 를 날아 간차가 0.0002 mm 수준이다.
고도 잔차 30.8 um 는 제공 코드의 ECEF→LLA 반복 근사에서 오는 상수 편차다.



값 하나가 작은 것보다, 간격을 바꿨을 때 정해진 기울기로 줄어드는 것이 더 강한 증거다.

17

발표 대본

결과 · 약 75초

검증을 세 갈래로 나눴습니다. 각각이 다른 종류의 실수를 잡기 때문입니다.

첫째, 해석해와 대조입니다. 답을 손으로 계산할 수 있는 항목을 골랐습니다. 대함은 초속 30미터 곱하기 60초 해서 1800미터, 대공은 12000미터여야 합니다. 차이가 10의 마이너스 8제곱, 마이너스 7제곱 미터입니다. 12킬로미터를 날아 0.0002밀리미터 차이입니다. 이 거리는 601개 점 사이의 거리를 전부 더해서 얻은 것이 아니라 직선거리가 아니라 실제로 지나간 길이입니다. 이건 스케일이 틀린 경우를 잡습니다.

둘째, 불변량 보존입니다. 변하면 안 되는 값을 봤습니다. 고도는 30.8마이크로미터, 속력은 601개 표본 모두 완전히 일치, 위도와 경도 편차는 0.1마이크로미터 이하입니다. 이건 방향이 틀린 경우를 잡습니다.

고도의 30.8마이크로미터는 짚고 갈 만합니다. 첫 스텝에서 한 번 오르고 그 뒤로는 변하지 않습니다. 적분 오차라면 시간에 비례해 커져야 하는데 안 커집니다. 원인을 찾아보니 제공받은 ECEF 에서 LLA 로 가는 함수의 5회 반복 근사였습니다.

셋째가 수렴 차수인데, 이게 제일 강한 증거라고 생각합니다. 오른쪽 그래프입니다.

대공 표적에 2G 로 20초 선회를 걸고 갱신 간격만 1초부터 0.01초까지 바꿔 가며 돌렸습니다. 양쪽 축이 로그인데, 파란 실선이 실측 오차, 회색 점선이 간격의 제곱에 비례하는 선입니다. 둘이 나란하고, 회귀 기울기가 2.002 나옵니다. 중점법이 설계대로 2차 정확도로 돌고 있다는 증거입니다.

과제 조건인 0.1초에서 오차가 1.36센티미터입니다. 레이더 거리 분해능이 수 미터인 걸 생각하면 없는 값이라고 봐도 됩니다.

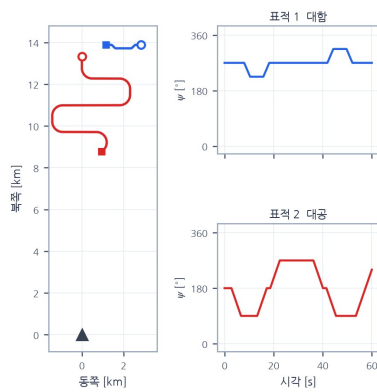
값 하나가 작은 건 우연일 수 있습니다. 그런데 간격을 바꿨을 때 정해진 기울기로 줄어들면 그건 구현이 의도한 방법을 정확히 따르고 있다는 뜻입니다.

03. 시뮬레이션 결과 및 검증

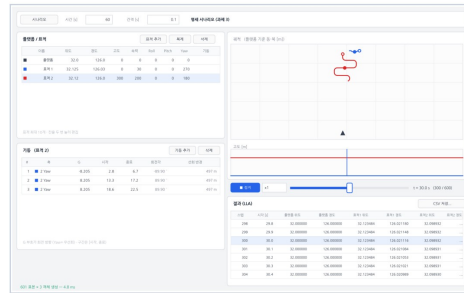
결과 ③ — 기동 시연과 운용 화면

초기값 · 시간 · 간격은 명세 그대로, 기동표 10줄만 추가했습니다.

기동 10줄을 얹은 60 s 궤적



TargetSim — 값을 바꿔 가며 확인하는 화면



고치면 곧바로 다시 돈다
표의 칸을 고치면 0.25 s 뒤 자동으로 다시 계산한다. 601 표본 × 3 객체가 4.8 ms 라 기다릴 일이 없다.

그림에서 끌어서 배치
시작점과 기수 손잡이를 마우스로 끌면 위도 · 경도 · Yaw 가 표에 반영된다. 회전각과 선회 반경은 G 로부터 계산.

결과는 표 · CSV · 그림
시각별 LLA 를 표로 보고 슬라이더로 되감고 CSV 로 내보낸다. 오늘 숫자가 전부 여기서 나온 CSV 다.

18

발표 대본

결과 · 약 55초

왼쪽이 기동을 얹은 결과입니다. 초기값도 시뮬레이션 시간도 갱신 간격도 명세 그대로이고, 기동표 10줄만 추가했습니다.

빨간색 대공 표적이 남쪽으로 내려가다가 동쪽으로 꺾고, 다시 남쪽, 다시 서쪽으로 훑고 내려갑니다. 오른쪽 두 그래프가 기수 방위각인데, 평평한 데가 직진, 기울어진 데가 기동입니다. 기동표에 적은 시각과 정확히 맞습니다.

회전 지점이 좌표로 주어져서 시각으로 바꾸는 작업을 따로 했습니다. Core 를 돌려 그 좌표에 닿는 시각을 찾았고, 회전하는 동안에도 표적이 계속 나아가니까 선회 반경만큼 미리 꺾어야 지정한 자리에서 수평이 됩니다.

가운데가 화면입니다. 값을 코드에 박아 놓고 다시 컴파일하는 게 불편해서 붙였습니다.

왼쪽 위 표에서 초기값을 고치면 0.25초 뒤에 자동으로 다시 계산합니다. 4.8밀리초면 끝나니 기다릴 일이 없습니다. 오른쪽 궤적에서는 시작점과 기수 손잡이를 마우스로 끌 수 있습니다. 아래 결과 표의 LLA 를 CSV 로 내보내는 데, 오늘 보여 드린 숫자가 전부 여기서 나온 CSV 입니다.

03. 시뮬레이션 결과 및 검증

정리

요구사항과 결과를 한 장에 놓고 보겠습니다.

과제 요구사항	구현	근거
표적 10 · 기동 30	고정 배열로 상한, 범위 밖은 오류 코드	범위 검사
위치 LLA · 속도 V_heading	ST_TargetInit, 방향은 자세각이 담당	검증 통과
기동 = G · 축 · 시작 · 종료	ST_TargetManeuver 4 필드, 구간 겹침 검사	검증 통과
ECEF 에서 상태 갱신	위치 · 속도 ECEF, 출력용 LLA 병행 보관	고도 잔차 30 μm
V_heading $\rightarrow$ Vx, Vy, Vz (추가 고민)	동체 $\rightarrow$ NED $\rightarrow$ ECEF, 회전 2 회	속력 완전 보존
제공 좌표변환 코드 사용	CoordinateTransform.c / matrixCalcLib.c 원본	무수정
결과를 LLA 로 출력하고 그림으로	표 · CSV · 궤적 · 고도 · 방위 그림	601 표본 출력

남겨 둔 한계

- Pitch $\pm 90^\circ$ 근처의 짐벌락 — 사원수로 전환하면 해소
- 힘을 풀지 않는 운동학 모델 — 탄도 표적에는 부적합
- 고도 잔차 30.8 μm 는 제공 코드의 ECEF $\rightarrow$ LLA 반복 근사

이어 붙일 수 있는 것

- 궤적에 측정 잡음을 얹으면 그대로 추적 필터 입력
- Part 2 의 좌표변환을 붙이면 안테나 기준 R · Az · El
- 표본 생성기를 바꾸면 RCS · 도플러까지 확장 가능

19

발표 대본

정리 · 약 60초

요구사항과 결과를 한 장에 놓고 보겠습니다.

표적 10개와 기동 30개는 고정 배열로 상한을 두고, 범위를 넘으면 오류 코드로 거부했습니다.

위치는 LLA, 속도는 V_heading 하나로 받았고 방향은 자세각이 담당합니다.

기동은 축, G, 시작, 종료 네 필드로 정의하고 구간 겹침까지 검사합니다.

상태 갱신은 ECEF 에서 하고 출력용 LLA 를 같이 보관합니다. 고도 잔차 30마이크로미터는 좌표변환 근사의 몫입니다.

추가 고민 항목이었던 속도 변환은 동체에서 NED, NED 에서 ECEF 로 두 번 회전했고, 속력이 완전히 보존되는 것을 601개 표본으로 확인했습니다.

좌표변환 코드는 제공받은 것을 무수정으로 썼고, 결과는 표와 CSV, 궤적과 고도와 방위 그림으로 냈습니다.

아래 왼쪽이 남겨 둔 한계입니다. 짐벌락, 힘을 풀지 않는 운동학 모델, 그리고 고도 잔차. 셋 다 원인을 알고 남겨 둔 것이고 해결 방법도 적어 뒀습니다.

오른쪽이 이어 붙일 수 있는 것입니다. 이 궤적에 측정 잡음만 얹으면 바로 추적 필터 입력이 되고, Part 2 에서 만든 좌표변환을 붙이면 안테나 기준 거리와 방위각, 고각이 나옵니다. 다음 과제로 이어지는 부분입니다.

감사합니다

질문 주시면 답변드리겠습니다

20

발표 대본

마무리 · 약 15초

이상입니다. 감사합니다.

질문 있으시면 답변드리겠습니다.